POLICYLENS

RBI Document Ingestion and Semantic Retrieval System

PROJECT REPORT

Submitted by	Jitesh Vishwakarma
Institution	ITM Skills University
Programme / Course	Agentic AI
Project	PolicyLens

Abstract

PolicyLens is a document-ingestion and semantic-retrieval system built around 30 public Reserve Bank of India (RBI) PDF documents. The corpus includes text-native, scanned and table-heavy material. The pipeline extracts page text with PyMuPDF, applies OCR as a fallback where page text is insufficient, cleans extracted text, creates chunks, enriches each chunk with document and page metadata, generates vector embeddings and stores the records in ChromaDB. The final embedding model is **intfloat/e5-base-v2**.

Two chunking approaches were compared using the same corpus, embedding model and 25-question evaluation set: fixed-size chunks (maximum 300 tokens with 60-token overlap) and structure-aware chunks (section-oriented, with long sections split at approximately 300 tokens). The final structure-aware collection contains 15,374 chunks; the fixed-size collection contains 19,341. Structure-aware chunking achieved Recall@3 of 0.88, Recall@5 of 0.96, Recall@10 of 1.00 and MRR of 0.7743. Fixed-size chunking achieved 0.72, 0.84, 0.92 and 0.7613 respectively.

A controlled ingestion timing test processed one 76-page scanned bulletin and stored 293 chunks in 12.467 seconds, including model loading. A separate query measurement was 217.66 ms for the ChromaDB query after query-embedding generation; it excludes embedding time. OCR spot checks on two pages found that headings and some labels were recovered, while chart details and small numbers remained unreliable. PolicyLens is a retrieval system: LLM-based answer generation is outside the implementation scope.

Keywords: PDF ingestion, OCR, semantic retrieval, chunking, embeddings, ChromaDB, RBI documents

1. Introduction and dataset

Regulatory PDFs contain rules, definitions, conditions, thresholds and tables. Locating a passage can be difficult when users phrase a query differently from the source. PolicyLens prepares PDFs for semantic search and supports metadata filters to narrow retrieval.

1.1 Objectives and scope

The project objectives are to ingest 30 public RBI PDFs; extract text with OCR fallback; clean and normalize text; build fixed-size and structure-aware chunks; attach document and page metadata; generate embeddings; store vectors in ChromaDB; and compare retrieval performance. The implementation covers ingestion, indexing and retrieval. It does not generate natural-language answers with an LLM, and its evaluation does not measure generated-answer correctness or citation faithfulness.

1.2 Corpus inventory

Document category	Count	Share
Text-native	19	63.3%
Scanned	6	20.0%
Table-heavy	5	16.7%
Total	30	100%

The corpus audit verified 30 manifest records and 30 corresponding PDFs. It reported no duplicate document IDs or filenames, missing PDFs, page-count mismatches or unregistered PDFs. The manifest is stored at **data/manifest.csv**. Chunk metadata includes fields such as document ID, title, issuer, issue date, page range, section title and source hash, where available.

2. System architecture and methodology

2.1 Extraction and OCR

PyMuPDF is used to extract text from text-native pages. Pages with insufficient extracted text can be sent to the OCR fallback, implemented with Tesseract through pytesseract. OCR renders pages at 300 DPI, preprocesses the image (grayscale, autocontrast and contrast enhancement), attempts orientation detection, and runs English OCR. This hybrid approach avoids OCR on pages where usable embedded text is already available, while providing a recovery path for scanned pages.

2.2 Cleaning

The cleaner normalizes line endings, removes control characters, joins words split by hyphenation across line breaks, normalizes spaces and tabs, reduces excessive blank lines and trims the result. It does **not** currently implement systematic repeated-header, footer or page-number removal; these remain a source of possible retrieval noise.

Example from the cleaner test: “Financial inclu-
sion is important.” becomes “Financial inclusion is important.” The test also normalized repeated spaces in “Banking Regulation” to “Banking Regulation.”

3. Chunking, metadata and vector indexing

3.1 Chunking strategies

Strategy	Implemented rule	Final chunk count
Fixed-size	Maximum 300 tokens; 60-token overlap. Average 299.8 tokens; maximum 300.	19,341
Structure-aware	Uses detected document structure and section boundaries; long sections are split at approximately 300 tokens.	15,374

Both final collections were built from the same 30-document corpus and use the same embedding model. Structure-aware chunking aims to keep related content together and reduce arbitrary splits at section boundaries. Fixed-size chunking provides more uniform passage lengths and overlap between adjacent chunks.

3.2 Metadata enrichment

Each chunk is associated with document identity and descriptive fields, including document ID, title, issuer, issue date, page start/end, section title and source hash where available. These fields support traceability and metadata filtering. Page-range metadata is particularly important for locating the source passage in the original PDF.

3.3 Embeddings and index

The final experiments use **intfloat/e5-base-v2**, a 768-dimensional embedding model. Document text is embedded with the passage prefix and search queries with the query prefix; embeddings are normalized. ChromaDB stores vectors, chunk text and metadata. The collection uses cosine distance as its similarity space. The CLI returns ranked passages with document and page metadata and supports top-k retrieval and metadata filters.

3.4 Operational behaviour

The ingestion process is designed to be rerunnable. A repeat structure-aware ingestion of the final corpus skipped all 30 already processed documents and added zero new chunks. The project also maintains separate collections for the two chunking strategies so that evaluation runs do not overwrite one another. The current implementation uses source hashes to identify changed source documents and replaces per-document records when a document needs reprocessing.

The report does not claim an automated superseded/withdrawn-circular lifecycle workflow. For production use, document status and effective dates should be stored explicitly; newer circulars should be versioned rather than silently overwriting historical records. Retrieval should be able to filter out withdrawn material by default while retaining it for historical queries.

4. Retrieval evaluation and results

4.1 Evaluation dataset and controls

The evaluation contains 25 questions with gold document/page annotations: 15 standard questions (Q01–Q15), five paraphrased questions (Q16–Q20) and five table-based questions (Q21–Q25). Both strategies use the same 30 PDFs, questions and embedding model. The evaluation output records question ID, rank, retrieved document ID, title, page range, similarity score and whether the retrieved result matches the gold document/page annotation.

4.2 Metrics

Recall@k is the proportion of questions for which a relevant document/page result appears in the first k results. **MRR** (Mean Reciprocal Rank) averages the reciprocal rank of the first relevant result across questions. These are retrieval metrics, not answer-generation metrics.

4.3 Aggregate results

Metric	Structure-aware	Fixed-size	Difference
Recall@3	0.8800 (88%)	0.7200 (72%)	+16 percentage points
Recall@5	0.9600 (96%)	0.8400 (84%)	+12 percentage points
Recall@10	1.0000 (100%)	0.9200 (92%)	+8 percentage points
MRR	0.7743	0.7613	+0.0130

Structure-aware chunking performed better on all four aggregate metrics in this test set. It achieved 100% Recall@10, meaning a gold document/page match appeared within the first ten results for all 25 questions. The MRR difference is relatively small, so the largest observed advantage is in recall at the tested cut-offs.

4.4 Table-based subset

For Q21, Q22 and Q23, fixed-size chunking ranked the relevant result first while structure-aware chunking ranked it second. Both strategies ranked relevant results first for Q24 and Q25. The subset contains only five questions, so it is too small to establish that fixed-size chunking is generally better for tables.

4.5 Interpretation

The evaluation is based on 25 manually specified questions and gold document/page annotations. Results depend on the coverage and quality of this test set. They should not be generalized to all RBI documents or all regulatory questions. A relevant page match also does not prove that the retrieved passage alone fully resolves a real-world compliance question.

5. Chunking comparison: question-level examples

The following examples show that the two strategies can rank different pages first. “Correct” means that the top result matched the evaluation's gold document/page annotation; it does not refer to the correctness of an LLM-generated answer.

Question	Structure-aware top result	Fixed-size top result
Q04 – Digital lending data collection	RBI_CREDIT_001, p. 23 — correct	RBI_CREDIT_001, pp. 18–19 — not correct
Q07 – Project finance disbursement	RBI_CREDIT_001, pp. 72–73 — not correct	RBI_CREDIT_001, p. 47 — correct
Q12 – Board-approved credit policy	RBI_CREDIT_001, pp. 14–16 — correct	RBI_CREDIT_001, p. 89 — not correct

Q04 and Q12 favour the structure-aware top result, while Q07 favours the fixed-size top result. The examples illustrate why aggregate metrics and per-question inspection are both useful: a strategy can perform better overall without winning every individual query.

5.1 Parsing and OCR quality observations

Two pages from the February 1952 RBI Bulletin were manually inspected as OCR spot checks; this is not a random ten-page sample and no word-error-rate or character-accuracy percentage was calculated.

Page	Observed extraction	Quality assessment
17	The page contained a chart/illustration. OCR returned some headings and labels, but also garbled text and symbols.	Poor for chart detail and small labels; not suitable for relying on exact values.
36	OCR recovered headings such as “WHOLESALE PRICES AND COST OF LIVING” and “INDUSTRIAL PRODUCTION,” plus some axis/base text.	Partial recovery; graph lines and several small numerical labels were not reliably recognized.

These checks show that OCR can recover some searchable context from scanned pages but may fail on charts, dense layouts and small numerals. Numerical values from scanned charts should be verified against the page image. A stronger quality-control process would sample at least ten pages across document types, compare OCR output with manually transcribed text, and record a measured error rate.

5.2 Parser and table limitations

The current pipeline uses page-level text extraction and OCR fallback. Complex tables, multi-column layouts and chart-based information can lose spatial relationships when represented as plain text. Possible mitigations include layout-aware extraction, table detection and structured table serialization, higher-resolution OCR for difficult pages, and manual verification of high-risk numerical content.

6. Efficiency and operational evidence

6.1 Controlled ingestion timing

Measurement	Observed result	Scope / qualification
Ingestion duration	12.467 seconds	One 76-page February 1952 scanned bulletin; includes model loading, extraction, chunking, embedding and storage.
Pages processed	76	One PDF; successful completion.
Chunks stored	293	Verified by querying the temporary ChromaDB test collection.
Failures	0	One document succeeded; zero failed.
Estimated time per 100 pages	16.40 seconds	Linear extrapolation from the 76-page run; not a direct 100-page benchmark.

A separate run over the scanned-PDF folder took 25 minutes 41.63 seconds before it was manually interrupted during OCR of the RBI Report on Currency and Finance 1959–60 (page 87). It completed five PDFs and stored 4,395 chunks in a temporary test collection, which was subsequently deleted. This is a partial-run observation, not the total ingestion time for the scanned corpus or all 30 documents.

6.2 Query latency and index footprint

Metric	Observed value	Interpretation
ChromaDB query latency	217.66 ms	One top-5 vector query after query embedding was generated; excludes embedding-model inference time.
Persistent ChromaDB directory	429 MB	Previously measured total for data/chroma_db across all then-existing collections; not the size of either final collection alone.
Final structure-aware collection	15,374 chunks	30-document corpus.
Final fixed-size collection	19,341 chunks	Same 30-document corpus.

The query latency is a single observed measurement, not a statistically summarized benchmark. A more complete latency study should report warm-up, embedding time, vector-search time, end-to-end time, query count, and median/p95 across a representative query set. The 429 MB figure is an aggregate directory measurement and should not be attributed to one collection.

6.3 Embedding cost

The project uses a locally loaded intfloat/e5-base-v2 model. No paid embedding API was used in the reported pipeline, and no monetary cost per 1,000 chunks was measured. The practical cost is local compute time and memory. A comparable cost figure would require recording hardware, model-loading overhead, batch size, throughput and energy or infrastructure pricing; therefore no unsupported currency estimate is reported.

6.4 Scaling to 100,000 documents

At 100,000 documents, ingestion should be distributed into bounded batches with checkpointing, retry/backoff and per-document status logs. Store source hashes and document versions to support incremental updates. Partition or shard the vector index by issuer, year or document family where useful; monitor index size, embedding throughput, query latency and recall as the corpus grows. Use a queue for OCR-heavy files so scanned PDFs do not block text-native ingestion. Run regression evaluations after model, parser or chunking changes.

7. Reliability, privacy, limitations and conclusion

7.1 Reliability and rerun behaviour

The final structure-aware collection contains 15,374 chunks and the fixed-size collection contains 19,341 chunks, each built from the same 30-document corpus. The recorded final ingestion runs completed all 30 documents with zero failures. A repeat structure-aware run skipped all 30 documents and added zero new chunks, providing evidence of idempotent rerun behaviour for an unchanged corpus.

The temporary timing collections used for performance checks were deleted after verification. They are not part of the final collections or the final chunk counts.

7.2 Limitations

The main limitations are: (1) evaluation uses 25 manually specified questions; (2) table-based evaluation contains only five questions; (3) OCR quality was manually inspected on two pages only, without a measured accuracy rate; (4) charts and small numerical labels can be misread by OCR; (5) repeated headers, footers and page numbers are not systematically removed by the current cleaner; (6) query latency is a single measurement that excludes query embedding; and (7) no paid-API cost per 1,000 chunks was measured. The system retrieves passages but does not generate or validate final compliance answers.

7.3 Privacy and access control considerations

The current corpus consists of public RBI PDFs. For deployment with non-public or internal regulatory documents, access control must be enforced before retrieval, not merely in the user interface. Documents and chunks should carry tenant or permission metadata, and every query should be filtered by the caller's authorization. Protect source files, manifests, logs and vector storage with least-privilege access, encryption at rest and in transit, retention controls, audit logging and secret management. Avoid placing confidential source text in unrestricted logs or external services without approval.

7.4 Future improvements

Recommended improvements include a ten-page or larger stratified parsing-quality review; quantitative OCR accuracy measurement; layout-aware table extraction; repeated-header/footer detection with safeguards; end-to-end query latency benchmarking; a documented version/status model for superseded and withdrawn circulars; expanded evaluation with more table and paraphrase questions; and authorization-aware retrieval tests. An LLM answer-generation layer with source-grounded citations could be added as a separate future component.

8. Conclusion

PolicyLens implements an end-to-end ingestion and semantic-retrieval workflow for 30 RBI PDFs using PyMuPDF, Tesseract OCR fallback, text cleaning, two chunking strategies, metadata enrichment, intfloat/e5-base-v2 embeddings and ChromaDB. On the 25-question evaluation set, structure-aware chunking was the stronger overall configuration, achieving Recall@3 of 88%, Recall@5 of 96%, Recall@10 of 100% and MRR of 0.7743. Fixed-size chunking remained better for selected individual and table-based queries. The measured efficiency figures are useful initial observations but should be expanded through repeatable, end-to-end benchmarks.

References and project artifacts

1. Reserve Bank of India. Public circulars, directions and related PDF documents included in the PolicyLens corpus (inventory: data/manifest.csv).
2. intfloat. *e5-base-v2* embedding model.
3. Chroma. Vector database used for embedding, text and metadata storage.
4. Project artifacts: ingest.py, search.py, src/, eval/questions.csv, eval/metrics_30docs.csv, eval/metrics_fixed_30docs.csv and corresponding retrieval logs.

Submission note: The report identifies the institution and course supplied for this project. Add any instructor name, student ID, submission date, declaration or citation-style requirements only if your assignment brief specifically requires them.